



FROM FIRST RUN TO LONG-TERM SERVICE

Website implementation guide

A beginner-to-advanced field guide for building, deploying, and maintaining the Rising Design and Construction website.

Django 6.1 · PostgreSQL · HTMX · Tailwind CSS · Cloudflare R2 · Render

THE OUTCOME

A website you can operate—not just launch.

This guide explains the starter project from local setup and admin editing through secure uploads, production deployment, database changes, backups, and recovery. Use it alongside the supplied project ZIP.

START	BUILD	OPERATE
Run the project locally; understand the folders and settings.	Connect PostgreSQL, R2, email, and deploy to Render.	Ship migrations safely, back up data, and monitor real inquiries.

Important: free hosting is a demo tier

The included code is production-oriented, but Render Free is not suitable for relying on real customer inquiry data. Free Postgres expires; use persistent PostgreSQL and backups for a business launch. See section 9.

01 · ORIENTATION

Start with the system map

You do not need to understand every Django feature at once. Follow the request path and learn each layer as it appears.

LAYER	ROLE IN THIS PROJECT	WHERE TO LOOK
Browser / templates	Shows pages; HTMX submits the form without replacing the whole page.	templates/; static/; assets/css/
Django views	Receives URLs, queries records, validates the request, selects a response.	core/views.py; inquiries/views.py
Models / PostgreSQL	Stores services, portfolio content, team, testimonials, and inquiries.	content/models.py; inquiries/models.py
Media storage	Stores public portfolio files separately from private customer attachments.	core/storage.py; settings/production.py
Operations	Builds CSS, collects static files, migrates the database, starts Gunicorn.	render.yaml; requirements.txt; package.json

How a consultation request travels

STEP	WHAT HAPPENS
1 · Visitor	Completes the labelled form and optionally chooses a PDF/image file.
2 · HTMX	Sends a normal CSRF-protected multipart POST; the page stays in place.
3 · Django	Validates service, contact details, size, extension, and actual file content.
4 · Save	Stores lead data in PostgreSQL and the attachment in private R2.
5 · Notify	Sends an email notification; if email fails, the saved inquiry remains in admin.
6 · Respond	Returns a success or validation partial for HTMX to swap into the form area.

One-time setup vs. ongoing work

One-time: create accounts, set secrets, deploy, and create an admin user. Ongoing: edit content in admin, review inquiries, back up PostgreSQL, and deploy code changes with reviewed migrations.

Version note: the starter targets Django 6.1.x and Python 3.12–3.14. Recheck the official supported versions when upgrading. [1]

02 · BEGINNER

Prepare your computer

Use Python 3.13 for the starter, plus Node.js 20 or newer for the production Tailwind build. Django 6.1 supports Python 3.12–3.14. [1]

A. Unzip, open the project folder, and create a virtual environment

```
cd rising-design-construction
python3.13 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
cp .env.example .env # Windows PowerShell: Copy-Item .env.example .env
```

A virtual environment isolates this website's Python packages from other projects on your machine. Activate it every time you work on this project.

B. Add a local secret and install dependencies

```
python -c "import secrets; print(secrets.token_urlsafe(64))"
# Paste the output into .env as SECRET_KEY=...
pip install -r requirements.txt
npm install
npm run build:css
```

Keep secrets out of Git

The `.env` file contains credentials. It is ignored by `.gitignore`; commit `.env.example` as a safe list of required variable names, never `.env`. Generate a different secret for production.

C. Check the local environment

- `.env.example` starts with `DEBUG=True`, local hosts, and console email. Replace the placeholder `SECRET_KEY`.
- If `DATABASE_URL` is unset, local development uses SQLite so you can start quickly. Production settings require PostgreSQL.
- With `USE_R2=False`, local file uploads go to `media`. Do not use that local path for Render production.

```
python manage.py check
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open `http://127.0.0.1:8000/`. Stop the server with `Ctrl+C`.

03 · BEGINNER

Make the first useful edits

The starter seeds your eight services. Use Django admin for day-to-day content updates rather than changing Python code.

URL	PURPOSE	FIRST CHECK
/	Home and highlighted services	Featured services appear after migration.
/services/	Service catalogue	Active services are listed.
/projects/	Case studies	Empty state appears until you add a project.
/about/	Company intro and team	Add team members in admin to populate this page.
/consultation/	HTMX inquiry form	Try valid, invalid, and file-upload submissions.
/admin/	Content editing and lead follow-up	Sign in with the superuser you created.
/health/	Basic deployment health response	Should return plain-text ok.

Add a case study in admin

- Choose a service, provide a unique title/slug, short summary, plain-text body, project date, and cover image.
- A case-study PDF is public by design. Upload only material approved for public display; inquiry attachments use private storage.
- Testimonials are client statements; never invent or publish quotes without permission. Team profiles and photos should be approved by the people shown.

What the form’s project dropdown does

The dropdown is populated from active Service records. If you add a new service in admin, it can become a project type without editing form choices in code. Deactivating a service removes it from new requests but retains existing inquiry history.

Demo recipient

The example notification recipient is `demo@example.com`. It is only a placeholder and cannot receive mail. Set your real recipient and a working sender before accepting public inquiries.

04 · BUILD

Understand the data model

Models are the durable business records. Choose field types to reflect the data and relationships you need to preserve.

RECORD	KEY DESIGN	WHY IT MATTERS
Service	Unique slug; active/featured flags; display order.	Slug gives a stable URL identity; active controls public pages and form choices.
CaseStudy	Required cover image; optional public PDF; service FK; date.	PROTECT prevents deleting a service still used by a case study.
Testimonial	Client, optional company/photo, quote, featured flag.	Optional company/photo avoids forcing data that may not exist.
TeamMember	Name, role, bio, optional photo, active/order.	Inactive members can be hidden without deleting history.
Inquiry	Name, email, phone text, service FK, message, private file, created time, status.	Phone is text, not a number; service relation keeps types consistent.

Relationships and delete behavior

- ``Inquiry.project_type → Service`` and ``CaseStudy.service → Service`` are foreign keys. The database enforces valid references.
- ``on_delete=PROTECT`` prevents accidental deletion of a service that existing business records need. Deactivate it instead.
- Slugs are useful for human-readable project URLs; titles can be edited without changing the record's database identity.
- Inquiry status is a workflow field (New, Reviewing, Contacted, Closed). Contact details and original message are read-only in admin.

When to change a model

If you add/change a database field, create a migration and commit it with the code. Admin content edits do not need migrations. Never manually alter production tables to “make the error go away.”

05 · BUILD

Form validation and HTMX

HTMX improves the interaction; Django remains the authority. Browser hints are helpful, but server-side validation protects stored data.

Upload checks

- The form caps inquiry attachments at 10 MiB and allows PDF, JPG/JPEG, PNG, and WebP. SVG is excluded because it can contain active content.
- The server checks extension and signature; Pillow verifies image structure/format. A browser-supplied MIME type is not trusted.
- The upload field uses the private storage alias. A valid extension alone is never treated as proof that a file is safe.

Why the form does not reload

The form POST includes Django's CSRF token and file data. On a valid HTMX request, Django returns a confirmation fragment; on invalid input, it returns the re-rendered form with errors. Both replace the contents of the form container. The form also works as a normal browser POST when JavaScript is unavailable.

Keep these safety properties

- Leave `CsrfViewMiddleware` enabled and keep `{% csrf_token %}` in the form.
- Django template auto-escaping should remain on. Do not apply `|safe` to user messages, quotes, or admin-entered descriptions.
- The honeypot is a light spam deterrent, not rate limiting. If spam becomes a problem, add rate limiting or a managed anti-abuse service.

Why validation errors return HTTP 200

HTMX normally does not swap most error-status responses. The starter renders field errors with status 200 so the form fragment is swapped automatically. The displayed errors—not the HTTP status—tell the visitor what to fix.

Longer-term scaling: if email delivery becomes slow or unreliable, store a notification job/outbox and send asynchronously. For this starter, the lead is saved first and email failure is logged rather than discarding the request.

06 · PRODUCTION

Settings, static files, and security

Production settings are a separate configuration, not a manual “remember to switch DEBUG off” step.

SETTING	LOCAL	PRODUCTION
DEBUG	True in development.	Must be False; production module refuses to start otherwise.
SECRET_KEY	Private local value.	Long generated secret in Render env; never commit it.
ALLOWED_HOSTS	localhost, 127.0.0.1.	Exact hostnames only; do not use *.
CSRF_TRUSTED_ORIGINS	Local origins.	Full origins such as <code>https://site.onrender.com</code> .
Database	SQLite by default.	PostgreSQL via DATABASE_URL.
Static files	Tailwind browser CDN in debug.	Minified Tailwind CSS built locally, then collected by WhiteNoise.

Tailwind and static files

The Tailwind browser CDN is convenient for local development but is documented for development use, not production. The deploy build compiles Tailwind to `static/css/site.css`; `collectstatic` gathers that file and the logo, and WhiteNoise serves static files with the Django app. [4]

CSRF, cookies, HTTPS, and proxy behavior

Production settings use Render’s forwarded HTTPS header, secure session/CSRF cookies, HTTPS redirect, and HSTS. These depend on configuring the correct Render host and HTTPS origin. Avoid changing them to “fix” a bad host/origin value.

Useful checks

Run `python manage.py check` locally. In production-like configuration, run `python manage.py check --deploy --settings=config.settings.production` after required environment variables are set. Review warnings instead of suppressing them blindly.

07 · PRODUCTION

Keep media durable with Cloudflare R2

Render's service filesystem is not the media archive. This project stores portfolio media and customer attachments in object storage.

BUCKET	ACCESS	STORE HERE
Public media	Public custom domain enabled.	Approved case-study covers and public case-study PDFs only.
Private inquiries	No public domain/access.	Customer drawings and specifications attached to consultation requests.

Setup sequence

- Create both buckets in Cloudflare R2. Keep public access disabled on the inquiry bucket.
- Create a limited R2 API token with only the object read/write access the site needs for these buckets.
- Configure a custom domain for the public bucket. Do not treat a temporary development URL as a long-term media domain.
- Set `USE\_R2=True`, account ID, access key, secret key, both bucket names, and public custom-domain hostname in Render.

```
R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_PUBLIC_BUCKET=rising-public-media
R2_PRIVATE_BUCKET=rising-private-inquiries
R2_PUBLIC_CUSTOM_DOMAIN=media.example.com
```

The project uses the R2 S3-compatible endpoint `https://<ACCOUNT_ID>.r2.cloudflarestorage.com` and region `auto`. `django-storages` signs private URLs with a short expiry; public media uses the public custom domain. [5]

Privacy rule

Do not put confidential incoming drawings in the public media bucket. Public case-study PDFs are public to anyone with the link. Make sure clients have authorized anything you publish.

08 · PRODUCTION

Email that works on your host

The notification recipient, sender identity, and delivery provider are separate settings. Test each one before launch.

VARIABLE	PURPOSE	EXAMPLE / NOTE
INQUIRY_NOTIFY_EMAIL	Where new leads are sent.	Replace <code>demo@example.com</code> with a monitored inbox.
DEFAULT_FROM_EMAIL	Verified sender identity.	Use a sender address/domain accepted by the provider.
RESEND_API_KEY	HTTPS API credential.	Set for the included Resend API path.
EMAIL_HOST, EMAIL_PORT	SMTP alternative.	For local/paid hosting; port/TLS must match provider.
EMAIL_HOST_USER, EMAIL_HOST_PASSWORD	SMTP credentials.	Store only in environment variables.

Local mode uses Django's console email backend: after a successful submission, inspect the runserver terminal. In production, the project uses the Resend HTTPS API when `RESEND_API_KEY` is set. Verify the sender domain with the provider and test delivery to the real recipient.

Render Free limitation

Render Free web services block outbound SMTP ports 25, 465, and 587. Use HTTPS email delivery (Resend is wired into this starter) or move to hosting that permits your SMTP provider. A valid SMTP password alone will not overcome a blocked network port. [2]

Failure behavior

The inquiry is saved before the notification is attempted. Email errors are logged; they do not delete the lead or falsely reject the visitor's request. Check Render logs and the email provider's activity/delivery records after a test submission.

09 · DEPLOYMENT

Deploy the app to Render

The Blueprint describes a free web service. Set the required environment variables before the first successful production build.

1. Push the project folder to a GitHub repository. Connect the repository to Render as a Blueprint and review `render.yaml`.
2. Configure PostgreSQL. `DATABASE_URL` is left for you to set. Use a persistent database for real customer records; Render Free Postgres expires after 30 days. [2]
3. Set exact host/origin values. Hosts are comma-separated hostnames; CSRF trusted origins are full HTTPS origins. Set all R2 and email variables too. `SECRET_KEY` is generated by the Blueprint.

Build command (also configured in the Blueprint):

```
npm install --no-audit --no-fund && npm run build:css
pip install -r requirements.txt
python manage.py collectstatic --noinput --settings=config.settings.production
python manage.py migrate --noinput --settings=config.settings.production
```

Start command:

```
gunicorn config.wsgi:application --bind 0.0.0.0:$PORT \
--workers 2 --threads 4 --timeout 90
```

After deploy, create the admin user using Render Shell if available, or run `createsuperuser` from a trusted machine using the production database URL. Never put the superuser password in source code. Test `/health/`, `/admin/`, one valid inquiry, a rejected file, private object access, and notification delivery.

Why Free is not long-term production

Render Free web services use an ephemeral filesystem; Free Postgres expires after 30 days; free web services block SMTP ports. R2 prevents media loss, but it does not make the database persistent. Use persistent PostgreSQL and backups for a live business site. [2]

Render's Django deployment guide documents its Django build/static workflow. [3]

10 · ADVANCED

Ship changes without losing data

Use Git for code history and Django migrations for database history. Content editors should use admin for records that are already modeled.

Normal code-change workflow

WHEN	DO THIS	WHY
Template/CSS change	Edit locally, run, inspect desktop/mobile, commit.	No database migration is needed.
Service/project/team content	Use Django admin.	Content is data, not source code.
Model field change	Edit model → makemigrations → inspect migration → test migrate → commit both.	Migration is the repeatable schema-change instruction.
Dependency change	Upgrade deliberately; run tests/checks; commit lock files.	Avoid unreviewed version drift.

Migration commands

```
python manage.py makemigrations
python manage.py sqlmigrate content 0003 # inspect SQL; use your actual migration name
python manage.py migrate
python manage.py test
```

Commit the generated migration with the model change. Deployment runs migrate; do not run makemigrations on Render. Prefer additive, backwards-compatible database changes when a previous app version may still be serving traffic.

Make builds repeatable

After `npm install` creates `package-lock.json`, commit it. For more deterministic asset builds, change the Render build step to `npm ci` once the lockfile is committed. For Python, review/lock transitive dependency versions on a regular schedule and test before deploying.

Release process

- Work on a branch; review the diff and migration; run tests; push to GitHub.
- Check Render build logs. If the build fails, fix the source/migration or environment and redeploy—do not delete production data to force it through.
- After deploy, smoke-test the changed page/form/admin and inspect logs. Keep the previous Git commit/release available for code rollback.

11 · ADVANCED

Operate, back up, and recover

For a business site, uptime is only one part of reliability. Leads, attachments, and the ability to restore them matter more.

Backups and recovery

- Enable automated PostgreSQL backups on the chosen database plan/provider. Confirm retention and restore procedures.
- Back up object storage separately if files are business-critical; database backups do not contain R2 objects.
- Schedule a restore drill: restore into a separate database/bucket and verify the app can read records and files. A backup that has never been restored is unproven.
- Define retention and deletion rules for customer drawings and contact details. Limit private R2 access to the application and authorized staff.

Routine operating cadence

CADENCE	CHECK
Each business day	Review new inquiries in admin; confirm the notification path is working.
Monthly	Review admin users, provider logs, failed requests, storage usage, and dependency notices.
Quarterly	Restore-test backups; audit bucket privacy, retention, account access, and production secrets.
Before each release	Run tests/checks; inspect migrations; back up before high-risk data changes.

When volume grows

The current form saves then sends email during the web request. This is intentionally simple for an early site. If delivery becomes slow or traffic rises, move notifications to a background queue/outbox and add rate limiting, monitoring, and an explicit retry policy. Keep the database record as the source of truth.

Admin security

Use individual accounts, strong unique passwords, least privilege, and a protected admin route/domain if practical. Do not share the superuser login. Review staff access when someone leaves.

12 · RUNBOOK

Troubleshooting and launch checklist

Start with Render build/runtime logs and the exact environment variable named in the error. Avoid changing security settings without understanding the cause.

SYMPTOM	LIKELY CAUSE	FIRST CHECK
400 / CSRF failure	Host/origin mismatch.	ALLOWED_HOSTS uses hostnames; CSRF uses full HTTPS origins.
500 on startup	Missing secret, DB, or R2 config.	Production settings intentionally fail fast; compare Render vars with <code>.env.example</code> .
No CSS	Tailwind build/static collection failed.	Check <code>npm run build:css</code> , generated <code>static/css/site.css</code> , and <code>collectstatic</code> logs.
Image/PDF broken	R2 bucket/domain/permissions issue.	Check bucket name, domain, token access, object path, and Render logs.
Inquiry saves; no email	Sender/recipient/API config or SMTP block.	Check <code>RESEND_API_KEY</code> , verified sender, provider events, and application logs.
Private file link fails	Expired signature or permissions.	Admin link expires; reload page; confirm private bucket stays private.

Go-live checklist

- ☐ Persistent PostgreSQL is configured and a restore-tested backup exists.
- ☐ R2 public/private buckets are separated; test inquiry attachment is not public.
- ☐ ``DEBUG=False``; secret generated; exact hosts and HTTPS CSRF origins set.
- ☐ Tailwind compiled and WhiteNoise serves static assets after deploy.
- ☐ Real sender and recipient verified; test email arrives; failure logs are visible.
- ☐ Admin user is secured; content, photos, PDF permissions, and privacy notices reviewed.
- ☐ Valid/invalid form submissions tested on desktop and mobile; check service dropdown and 10 MiB rejection.
- ☐ Privacy/retention policy and incident contact are documented for the team.

Short glossary

Migration: versioned database schema change. Static: CSS/JS/logo files collected for serving. Media: admin/client uploads. Foreign key: a database link between records. HTMX: sends form requests asynchronously and swaps HTML fragments. Environment variable: configuration/secret supplied outside source code.

References: [1] Django 6.1 release notes. [2] Render Free limits. [3] Render Django deploy. [4] Tailwind Play CDN. [5] django-storages for Cloudflare R2.